

AI-Assisted coding Lab test=01

Name: Pothunuri Akshitha

Date:1/September/2025

Roll no: 2403A510C2

Day: Monday

Batch no: 05

Department : CSE 2nd year

Q1. Zero-shot Classification

- **Task 1: Write a zero-shot prompt to classify sentiment without any examples.**

Prompt:

"""

Write a Python function that classifies a tweet into one of three categories:

"Positive", "Negative", or "Neutral".

Do not use any training examples.

Use a simple rule-based or zero-shot approach (e.g., with keyword matching or a pre-trained sentiment analysis model).

The function should take a string (tweet) as input and return the sentiment label.

"""

Code generated:

```
first.py x
C:\Users\akshi> first.py > ...
1 def classify_tweet_sentiment(tweet: str) -> str:
2     positive_keywords = ['amazing', 'great', 'love', 'happy', 'good', 'fantastic', 'excellent', 'awesome', 'wonderful']
3     negative_keywords = ['bad', 'terrible', 'hate', 'awful', 'worst', 'sad', 'disappointing', 'poor', 'horrible']
4
5     tweet_lower = tweet.lower()
6     if any(word in tweet_lower for word in positive_keywords):
7         return "Positive"
8     elif any(word in tweet_lower for word in negative_keywords):
9         return "Negative"
10    else:
11        return "Neutral"
12
13    # Example usage
14    tweet = "I just tried the new café in town, and it was amazing!"
15    sentiment = classify_tweet_sentiment(tweet)
16    print(sentiment)
```

Output:

```
[Running] python -u "c:\Users\akshi\first.py"
Positive

[Done] exited with code=0 in 0.094 seconds
```

Observation:

The program defines a function `classify_tweet_sentiment(tweet: str) -> str` that classifies the sentiment of a given tweet.

It uses a **rule-based approach**:

- If the tweet contains any word from the `positive_keywords` list, it is classified as **"Positive"**.
- If the tweet contains any word from the `negative_keywords` list, it is classified as **"Negative"**.
- If neither is found, it defaults to **"Neutral"**.

The input tweet is converted to **lowercase** to avoid case sensitivity issues when matching keywords.

For the example input:

`tweet = "I just tried the new café in town, and it was amazing!"`

The word **"amazing"** is found in the `positive_keywords` list.

Therefore, the function returns **"Positive"**, which is printed as the output.

✅ **Final Outcome:** The code successfully classifies the given tweet sentiment as **Positive**, showing that the keyword-based sentiment detection works as expected.

Task 2:

Create a scenario where an AI assistant needs to help a student solve math problems.

(a) **Without context**

Prompt 1:

Write a Python program that takes a math problem as input (for example: "12 * 8") and outputs the solution.

Solve the problem step by step in code (e.g., parse the expression and evaluate it).

Make sure the program prints both the step-by-step explanation and the final answer.

Code generated:

```
first.py X
> Us Close (Ctrl+F4) first.py > ...
1 import re
2
3 def solve_math_problem(problem: str):
4     # Remove spaces
5     problem = problem.replace(' ', '')
6     # Match numbers and operator
7     match = re.match(r'(\d+)([+|-|*|/])(\d+)', problem)
8     if not match:
9         print("Invalid input format. Please enter like '12 * 8'.")
10        return
11
12    num1, operator, num2 = match.groups()
13    num1, num2 = int(num1), int(num2)
14
15    print(f"Step 1: Identify the numbers and operator.")
16    print(f"Number 1: {num1}")
17    print(f"Operator: {operator}")
18    print(f"Number 2: {num2}")
19
20    print("Step 2: Perform the calculation.")
21    if operator == '+':
22        result = num1 + num2
23        print(f"{num1} + {num2} = {result}")
24    elif operator == '-':
25        result = num1 - num2
26        print(f"{num1} - {num2} = {result}")
```

```

first.py
C:\Users> Users > akshi > first.py > ...
3  def solve_math_problem(problem: str):
26      print(f"{num1} - {num2} = {result}")
27      elif operator == '*':
28          result = num1 * num2
29          print(f"{num1} * {num2} = {result}")
30      elif operator == '/':
31          result = num1 / num2
32          print(f"{num1} / {num2} = {result}")
33      else:
34          print("Unknown operator.")
35          return
36
37      print(f"Final Answer: {result}")
38
39      # Example usage
40      problem = "12 * 8"
41      solve_math_problem(problem)

```

Output:

```

Step 1: Identify the numbers and operator.
Number 1: 12
Operator: *
Number 2: 8
Step 2: Perform the calculation.
12 * 8 = 96
Final Answer: 96

```

Observation:

The program defines a function `solve_math_problem(problem: str)` that can solve simple arithmetic problems with two operands and one operator (+, -, *, /).

The function works as follows:

1. Input processing: Removes spaces from the given problem string.
2. Pattern matching: Uses a regular expression (`re.match`) to extract the first number, operator, and second number.

3. Validation: If the input does not match the expected format, it prints an error message.
4. Step-by-step explanation: Prints the numbers and operator identified.
5. Computation: Performs the correct arithmetic operation depending on the operator.
6. Final Output: Displays both the calculation and the final answer.
7. ☒ **Final Outcome:** The code successfully interprets the arithmetic problem " 12×8 ", explains the steps, and computes the correct result (**96**) with a clear explanation

Prompt 2:

(b) With detailed context:

You are building an AI tutor for a 7th-grade student learning algebra.

The program should solve basic algebra problems step by step and explain each step clearly in simple language.

Write a Python function that takes a linear equation (e.g., " $3x + 7 = 22$ "),

solves for x , and prints the explanation step by step

(like isolating the variable, subtracting, dividing, etc.)

along with the final answer.

Code generated:

```
> Users > akshi > first.py > ...
1  import re
2
3  def solve_linear_equation(equation: str):
4      print("Let's solve the equation step by step!")
5      print(f"Equation: {equation}")
6
7      # Extract coefficients using regex
8      match = re.match(r'(\d+)x\s*([+-]\s*\d+)\s*=\s*(\d+)', equation.replace(' ', ''))
9      if not match:
10         print("Sorry, I can only solve equations like '3x + 7 = 22'.")
11         return
12
13     a = int(match.group(1))
14     b = int(match.group(2).replace(' ', ''))
15     c = int(match.group(3))
16
17     print(f"\nStep 1: The equation is {a}x + {b} = {c}")
18     print("We want to find the value of x.")
19
20     print(f"\nStep 2: Subtract {b} from both sides to isolate the term with x.")
21     right_side = c - b
22     print(f"{a}x + {b} - {b} = {c} - {b}")
23     print(f"{a}x = {right_side}")
24
25     print(f"\nStep 3: Divide both sides by {a} to solve for x.")
```

```

26     x = right_side / a
27     print(f"{a}x / {a} = {right_side} / {a}")
28     print(f"x = {x}")
29
30     print(f"\nFinal Answer: x = {x}")
31
32     # Example usage
33     solve_linear_equation("3x + 7 = 22")

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  FORKS
Let's solve the equation step by step!
Equation: 3x + 7 = 22

Step 1: The equation is 3x + 7 = 22
We want to find the value of x.

Step 2: Subtract 7 from both sides to isolate the term with x.
3x + 7 - 7 = 22 - 7
3x = 15

Step 3: Divide both sides by 3 to solve for x.
3x / 3 = 15 / 3
x = 5.0

Final Answer: x = 5.0

```

Observation:

The program defines a function `solve_linear_equation(equation: str)` that solves simple linear equations in the form $ax + b = c$.

It works as follows:

1. Input equation display: Prints the original equation.
2. Regex extraction: Uses a regular expression to parse values of:
 - $a \rightarrow$ coefficient of x
 - $b \rightarrow$ constant term

- $c \rightarrow$ RHS value
- 3. Validation: If the equation does not match the pattern, it shows an error message.
- 4. Step-by-step solving:
 - Prints the equation in standard form: $ax + b = c$.
 - Subtracts b from both sides: $ax = c - b$.
 - Divides both sides by a to solve for x .
- 5. Final Output: Prints the computed value of x .

Q2. One-shot vs Few-shot

Task 1: Write:

Prompt 1:

Write a Python program that classifies tweets as "Positive", "Negative", or "Neutral".

Use a one-shot learning approach: provide exactly one example of input and output before classifying new tweets.

Example:

Input: "I love this new phone, it's amazing!"

Output: Positive

Now, implement a function `classify_tweet(tweet: str)` that predicts sentiment for any new tweet using this one-shot setup.

"""

Code generated:

```

> Users > akshi > first.py > ...
1  def classify_tweet(tweet: str) -> str:
2      # One-shot example
3      example_input = "I love this new phone, it's amazing!"
4      example_output = "Positive"
5
6      # Use keywords from the example to guide classification
7      positive_keywords = ['love', 'amazing']
8      negative_keywords = ['hate', 'terrible', 'bad', 'awful']
9
10     tweet_lower = tweet.lower()
11     if any(word in tweet_lower for word in positive_keywords):
12         return "Positive"
13     elif any(word in tweet_lower for word in negative_keywords):
14         return "Negative"
15     else:
16         return "Neutral"
17
18     # One-shot example
19     print('Input: "I love this new phone, it's amazing!"')
20     print("Output:", classify_tweet("I love this new phone, it's amazing!"))
21
22     # Classify a new tweet
23     new_tweet = "The weather is bad today."
24     print('Input:', f'{new_tweet}')
25     print("Output:", classify_tweet(new_tweet))

```

Output:

```

[Done] exited with code=0 in 0.111 seconds

[Running] python -u "c:\Users\akshi\first.py"
Input: "I love this new phone, it's amazing!"
Output: Positive
Input: "The weather is bad today."
Output: Negative

[Done] exited with code=0 in 0.093 seconds

```

Observation:

- The program defines a function `classify_tweet(tweet: str)` to classify tweets into sentiment categories using a one-shot learning approach.
- In this setup, one example is provided:
 - Input: "I love this new phone, it's amazing!"
 - Output: "Positive"
- Based on this single example, the program builds keyword lists:
 - Positive keywords: ['love', 'amazing']
 - Negative keywords: ['hate', 'terrible', 'bad', 'awful']
- The input tweet is converted to lowercase for keyword matching.
- Classification logic:
 - If a positive keyword is found → returns "Positive".
 - If a negative keyword is found → returns "Negative".
 - Otherwise → returns "Neutral".

Execution Results

1. For the example tweet:
 - Input: "I love this new phone, it's amazing!"
 - Contains "love" and "amazing" → classified as Positive.
2. For a new tweet:
 - Input: "The weather is bad today."
 - Contains "bad" → classified as Negative.

Prompt2:

b) Few-shot Prompt (3–4 examples)

"""

Write a Python program that classifies tweets as "Positive", "Negative", or "Neutral".

Use a few-shot learning approach: provide 3–4 examples of input and output before classifying new tweets.

Examples:

Input: "I love this new phone, it's amazing!" → Output: Positive

Input: "This is the worst service ever." → Output: Negative

Input: "I am going to the store later." → Output: Neutral

Input: "The weather is fantastic today!" → Output: Positive

Now, implement a function `classify_tweet(tweet: str)` that predicts sentiment for any new tweet using this few-shot setup.

Code generated:

```
def classify_tweet(tweet: str) -> str:
    # Few-shot examples
    examples = [
        {"input": "I love this new phone, it's amazing!", "output": "Positive"},
        {"input": "This is the worst service ever.", "output": "Negative"},
        {"input": "I am going to the store later.", "output": "Neutral"},
        {"input": "The weather is fantastic today!", "output": "Positive"}
    ]

    # Build keyword lists from examples
    positive_keywords = ['love', 'amazing', 'fantastic']
    negative_keywords = ['worst', 'bad', 'terrible']
    neutral_keywords = ['store', 'going', 'later']

    tweet_lower = tweet.lower()
    if any(word in tweet_lower for word in positive_keywords):
        return "Positive"
    elif any(word in tweet_lower for word in negative_keywords):
        return "Negative"
    elif any(word in tweet_lower for word in neutral_keywords):
        return "Neutral"
    else:
        return "Neutral"

# Few-shot examples
examples = [
    "I love this new phone, it's amazing!",
    "This is the worst service ever.",
    "I am going to the store later.",
    "The weather is fantastic today!"
]
outputs = [classify_tweet(e) for e in examples]
```

```

33 for inp, out in zip(examples, outputs):
34     print(f'Input: "{inp}"')
35     print(f'Output: {out}')
36
37 # Classify a new tweet
38 new_tweet = "I had a fantastic day at the park."
39 print(f'Input: "{new_tweet}"')
40 print(["Output:", classify_tweet(new_tweet)])

```

Output:

```

[Running] python -u "c:\Users\akshi\first.py"
Input: "I love this new phone, it's amazing!"
Output: Positive
Input: "This is the worst service ever."
Output: Negative
Input: "I am going to the store later."
Output: Neutral
Input: "The weather is fantastic today!"
Output: Positive
Input: "I had a fantastic day at the park."
Output: Positive

```

Observation:

- The program defines a function **classify_tweet(tweet: str)** that classifies tweets into **Positive, Negative, or Neutral** using a **few-shot learning approach**.
- Unlike the one-shot version, here **multiple examples (3–4)** are provided, covering all three classes:
 - "I love this new phone, it's amazing!" → Positive
 - "This is the worst service ever." → Negative
 - "I am going to the store later." → Neutral
 - "The weather is fantastic today!" → Positive
- From these examples, the program builds **keyword lists**:
 - **Positive**: ['love', 'amazing', 'fantastic']
 - **Negative**: ['worst', 'bad', 'terrible']
 - **Neutral**: ['store', 'going', 'later']

- The input tweet is converted to **lowercase**, and classification is based on keyword matching.

Execution Results

1. "I love this new phone, it's amazing!" → contains love, amazing → **Positive**
2. "This is the worst service ever." → contains worst → **Negative**
3. "I am going to the store later." → contains going, store, later → **Neutral**
4. "The weather is fantastic today!" → contains fantastic → **Positive**

✓ The outputs **match the expected examples**.

5. New tweet: "I had a fantastic day at the park." → contains fantastic → **Positive**

Task 2:

Compare outputs on the same set of tweets and explain the difference.

Comparing the outputs:

Comparison table:

Tweet	One-shot Output	Few-shot Output	Observation
"I love this new phone, it's amazing!"	Positive	Positive	Both models correctly identify positive sentiment because keywords love and amazing are explicitly included.
"This is the worst service ever."	Neutral	Negative	One-shot fails (no negative keywords learned), while few-shot succeeds because worst was seen in examples.
"I am going to the store later."	Neutral	Neutral	Both predict Neutral, though few-shot is more confident since it has neutral examples.
"The weather is fantastic today!"	Neutral	Positive	One-shot misclassifies because it doesn't know fantastic; few-shot succeeds since fantastic was in training examples.

Tweet	One-shot Output	Few-shot Output	Observation
"I had a fantastic day at the park."	Neutral	Positive	One-shot misses the positive tone, but few-shot correctly classifies using keyword fantastic.

Code generated:

```

Users > akshi > first.py > ...
def classify_tweet_one_shot(tweet: str) -> str:
    # One-shot example
    positive_keywords = ['love', 'amazing']
    negative_keywords = ['hate', 'terrible', 'bad', 'awful']

    tweet_lower = tweet.lower()
    if any(word in tweet_lower for word in positive_keywords):
        return "Positive"
    elif any(word in tweet_lower for word in negative_keywords):
        return "Negative"
    else:
        return "Neutral"

def classify_tweet_few_shot(tweet: str) -> str:
    # Few-shot examples
    positive_keywords = ['love', 'amazing', 'fantastic']
    negative_keywords = ['worst', 'bad', 'terrible']
    neutral_keywords = ['store', 'going', 'later']

    tweet_lower = tweet.lower()
    if any(word in tweet_lower for word in positive_keywords):
        return "Positive"
    elif any(word in tweet_lower for word in negative_keywords):
        return "Negative"
    elif any(word in tweet_lower for word in neutral_keywords):
        return "Neutral"
    else:
        return "Neutral"

```

```

9
0 tweets = [
1     "I love this new phone, it's amazing!",
2     "This is the worst service ever.",
3     "I am going to the store later.",
4     "The weather is fantastic today!",
5     "I had a fantastic day at the park."
6 ]
7
8 print("| Tweet                                     | One-shot Output | Few-shot Output |")
9 print("| ----- | ----- | ----- |")
0 for tweet in tweets:
1     one_shot = classify_tweet_one_shot(tweet)
2     few_shot = classify_tweet_few_shot(tweet)
3     print(f"| {tweet:<40} | | {one_shot:<15} | | {few_shot:<15} | |")

```

Output:

```

[Running] python -u "c:\Users\akshi\first.py"
| Tweet                                     | One-shot Output | Few-shot Output |
| ----- | ----- | ----- |
| I love this new phone, it's amazing!     | Positive        | Positive        |
| This is the worst service ever.           | Neutral         | Negative        |
| I am going to the store later.            | Neutral         | Neutral         |
| The weather is fantastic today!           | Neutral         | Positive        |
| I had a fantastic day at the park.        | Neutral         | Positive        |

[Done] exited with code=0 in 0.095 seconds

```

Observation:

The one-shot classifier was able to correctly identify positive sentiment

when the tweet closely matched the given example. However, it failed to classify negative and some positive tweets correctly because its keyword coverage was limited to only the one example.

The few-shot classifier, on the other hand, performed better since it had multiple examples spanning all three classes (Positive, Negative, Neutral). This gave it richer keyword lists and broader context, allowing it to classify tweets more accurately.

For example, in the case of the tweet *"This is the worst service ever"*, the one-shot model returned Neutral (incorrect), while the few-shot model correctly returned Negative. Similarly, tweets with the word *"fantastic"* were missed by the one-shot classifier but correctly identified as Positive by the few-shot classifier.

Both models correctly identified simple neutral tweets like *"I am going to the store later"*.

